Memoization

Storing the result of expensive function calls and returning the cached result when the same inputs occur again.

It avoids recalculating the same thing.

Simple Example (Without Memoization)

```
function square(n) {
  console.log("Calculating...");
  return n * n;
}

console.log(square(5));
console.log(square(5));
```

Output:

Calculating...
25
Calculating...
25

🧠 2️⃣ **Add Manual Cache**

```
function square(n) {
  const cache = {};

  if (cache[n]) {
    return cache[n];
  }

  console.log("Calculating...");
  cache[n] = n * n;
  return cache[n];
}
```

🧠 3️⃣ Fix It Using Closure

```
function memoizedSquare() {
const cache = {};

return function(n) {
if (cache[n] !== undefined) {
console.log("From cache");
return cache[n];
}

console.log("Calculating...");
cache[n] = n * n;
return cache[n];
};
}

const square = memoizedSquare();

square(5);
square(5);
```

4️⃣ Generic Memoize Function (Important)

We don’t want to write custom wrapper every time.

So we build:

```
function memoize(fn) {
  const cache = {};

  return function(...args) {
    const key = JSON.stringify(args);
```

```
    if (cache[key]) {
      console.log("From cache");
      return cache[key];
    }

    console.log("Calculating...");
    const result = fn(...args);
    cache[key] = result;
    return result;
  };
}
```

Why JSON.stringify(args)?

Because:

- **args is an array**
- **Object keys must be strings**
- **So we convert arguments into a string key**

When NOT To Use Memoization

Very important.

Don’t memoize:

- **Functions with side effects**
- **Functions depending on changing external state**
- **Large input objects (memory explosion)**
- **Functions returning different outputs for same input (like Date.now)**

Memoization assumes:

Same input → Same output

This is pure function assumption.